

Gauge — Karar ve Trade-off Raporu

AI Destekli Senior .NET Mülakatı • Ingest servisi (Gauge.Ingest) Kapsam: Aday dokümanındaki problemlerin çözümü, alınan kararlar, trade-off'lar ve doğrulama sonuçları.

1. Yönetici Özeti

Monolitik ASP.NET Core ingest servisi, senkron aggregation ve retry kaynaklı veri bütünlüğü sorunları yaşıyordu. Sistem, mevcut mimari korunarak adım adım production-grade hale getirildi. Dokümandaki **kodlama görevleri (Görev A + Görev B) tam** karşılandı; gözlenen 5 semptomdan **4'ü tam**, 1'i (rapor/ingest DB izolasyonu) **bilinçli olarak kısmi/ertelenmiş** durumda. Tüm testler yeşil (**12/12**), SQL Server migration'ları doğrulandı.

2. İdempotency Maddesi Doğrulaması (Görev B)

Doküman şunu istiyor: `*"Aynı batchId ikinci kez geldiğinde API, ilk seferdeki sonucu döndürmeli ve DB'ye yazmamalı. batchId collector tarafından üretilen GUID olacak."*`

Kodda doğrulanan davranış:

- Kendi GUID üretmiyoruz: ``var batchId = request.BatchId;`` — fallback GUID kaldırıldı.
- batchId zorunlu: Endpoint ``request.BatchId == Guid.Empty`` ise ``400 BadRequest`` döndürüyor.
- Tekrarda yazma yok: ``ProcessedBatch`` içinde ``(TenantId, BatchId)`` bulunursa DB'ye yazmadan ilk sonucu (``ToResult(processed)``) döndürüyor.
- Eşzamanlı garanti: ``(TenantId, BatchId)`` unique index + transaction; yarış durumunda ``DbUpdateException`` yakalanıp mevcut sonuç döndürülüyor. DB-backed olduğu için 2 VM'de de geçerli.

Sonuç: Madde **doğru** implemente edilmiş.

3. Gözlenen Semptomlar → Ne, Nasıl Çözdük

#	Dokümandaki Semptom	Durum	Çözüm ve Yöntem
1	Her saat başı (:00/:15/:30/:45) latency 20-30 sn; collector timeout → tekrar gönderim	Çözüldü	Saatlik toplam hesaplama request içinden çıkarıldı. Ingest sadece <code>`DirtyHour`</code> işaretliyor; arka planda <code>`AggregationWorker`</code> (BackgroundService) toplamları hesaplıyor. Request süresi kısaldı.
2	Reading tablosunda mükerrer kayıt; raporlarda tüketim şişmesi	Çözüldü	Zorunlu <code>`batchId`</code> + <code>`ProcessedBatch`</code> üzerinde <code>`(TenantId, BatchId)`</code> unique index + tek transaction. Aynı batch ikinci gelişte yazmadan ilk sonucu döndürüyor.
3	Geç gelen okumalar; ilgili saatin toplamı yanlış kalıyor	Çözüldü	Geç okuma da o saati "dirty" yapıyor; worker saati baştan <code>`SUM`</code> ile yeniden hesaplıyor (delta eklemek yerine tam yeniden hesap → doğru sonuç).
4	Ay sonu raporlama ile ingest birbirini yavaşlatıyor	Kısmi (ertelendi)	Aggregation ayrıştırıldı, raporlama sorguları için index'ler eklendi. Ancak tek SQL Server instance korunuyor; read replica / CQRS bilinçli olarak sonraya bırakıldı.
5	En büyük tenant (yükün)	Çözüldü (1. seviye)	Tenant başına (API key

	%40'ı) diğer 39 tenant'ı etkiliyor		partition) concurrency rate limiter; limit dolunca `429 + Retry-After`. Kaynak tüketimi tenant bazında izole edildi.
--	------------------------------------	--	--

4. Kodlama Görevleri (Bölüm 2)

Görev A — Code Review (tespit + fix):

- Senkron aggregation (request içinde) → arka plan worker'a taşındı.
- Idempotency yok → `batchId` + unique index + transaction eklendi.
- Thread-unsafe `DbContext` (paralel işleme) → sıralı işleme + `AddRange`.
- In-memory quota (2 VM'de tutarsız, restart'ta kaybolur) → DB-backed quota.
- `Thread.Sleep` (host'u bloklar) → `Task.Delay(ct)`.
- Transaction eksikliği → Reading + ProcessedBatch + DirtyHour tek transaction.

Görev B — Idempotency Feature (kabul kriterleri karşılandı):

- `IdempotencyTests` Skip kaldırıldı ve geçiyor.
- Eşzamanlı aynı `batchId`: unique index + transaction retry ile garanti.
- Mevcut testler geçmeye devam ediyor. Toplam 12/12 test başarılı.

5. Alınan Kararlar ve Trade-off'lar

Karar	Trade-off (ne kazandık / ne kabul ettik)	Değerlendirme
Senkron → arka plan aggregation	Kazanç: düşük ingest latency, timeout azalması. Kabul: eventual consistency — rapor birkaç saniye gecikebilir.	Doğru
Idempotency = DB unique index + transaction (in-memory cache değil)	Kazanç: 2 VM ve restart'a dayanıklı gerçek garanti. Kabul: her ingest'te ek DB kontrolü/yazımı.	Doğru
Rate limiter = process-local concurrency	Kazanç: noisy-neighbor koruması, basit. Kabul: cross-VM global değil (en büyük tenant teorik olarak 2xlimit'e ulaşabilir); dağıtık limit (Redis) ertelendi.	Doğru (dokümente)
DB-backed tenant quota	Kazanç: çok VM tutarlılığı + restart hayatta kalır. Kabul: quota okuması DB sorgusu.	Doğru
Distributed worker lease (LeaseId/LeaseUntil)	Kazanç: iki VM worker'ı aynı DirtyHour'u işlemez, çift hesap önlenir. Kabul: lease süresi/atomik update yönetimi.	Doğru
DirtyHour dedup (unique index)	Kazanç: aynı meter/saat için tek aggregation işi. Kabul: ekleme sırasında dedup kontrolü.	Doğru
SQLite (test/local) + SQL Server (prod) ayrı context & migration	Kazanç: provider farkını gizlemeden gerçek prod şeması. Kabul: iki migration seti bakımı.	Doğru
Reporting AsNoTracking geri alındı	İlgili branch'e ait olduğu için geri alındı; fonksiyonel etkisi yok.	Doğru
Collector durable spool / retry (backoff+jitter)	Server tarafı hazır (idempotency, 429+Retry-After). Collector kapsam dışı olduğu için implemente edilmedi, handoff notu bırakıldı.	Kapsam dışı

6. Bilerek Şimdi Çözmediklerimiz (Bölüm 3 — Riskler)

- Rapor/ingest DB izolasyonu: Tek SQL Server instance sürüyor. Read replica / CQRS ertelendi. Risk: ay sonu yoğunluğunda okuma-yazma çekişmesi devam edebilir (semptom #4'ün tam çözümü).

- Cross-VM global rate limit: Her VM kendi limitini uyguluyor. Gerçek global limit için dağıtık sayaç (ör. Redis) gerekir.
- Büyük batch işleme: ~50.000 okuma tek transaction'da sıralı işleniyor; çok büyük batch'lerde transaction süresi/bellek izlenmeli (gerekirse chunk'lama).
- Collector contract: Bu repo server tarafıdır; collector'ın retry/durable davranışı production öncesi doğrulanmalı (README'de dokümanite edildi).

7. Doğrulama Sonuçları

Kontrol	Sonuç
`dotnet test` (tüm suite)	12 / 12 başarılı
SQL Server migration (local doğrulama)	Uygulandı, bekleyen model değişikliği yok
Linter	Hata yok
Idempotency (eşzamanlı aynı batch)	Tek yazım garanti edildi
Tenant izolasyonu (meter/usage/hourly)	Doğrulandı

8. Sonuç

Dokümandaki tüm kodlama görevleri eksiksiz karşılandı ve gözlenen semptomların büyük çoğunluğu çözüldü. Ertelenen maddeler rastgele değil; dokümanın "hangi problemi bilerek şimdi çözmüyorsunuz?" beklentisine uygun, açıkça işaretli ve gerekçeli kararlardır. Tek açık mimari başlık, semptom #4'ün tam çözümü olan raporlama/ingest veri izolasyonudur (read replica / CQRS) ve bir sonraki adım olarak önerilir.